

[object Object]

FlowChat

API

Docker

Compose

MongoDB

4.4

WhatsApp

Baileys

 Documentação oficial para deploy da API multi-sessão WhatsApp mais avançada

Deploy

EasyPanel

Docker

Ready

AI

Powered

[object Object]

[object Object]

-  [Visão Geral](#)
-  [Deploy Rápido](#)
-  [Pré-requisitos](#)

[object Object]

-  [Docker Compose](#)
-  [Variáveis de Ambiente](#)
-  [Estrutura do Projeto](#)

[object Object]

-  [Deploy no EasyPanel](#)

-  [Configuração de Domínio](#)
-  [Segurança e Produção](#)

[object Object]

-  [Monitoramento](#)
-  [Troubleshooting](#)
-  [API Endpoints](#)

[object Object]

-  [Configuração IA](#)
-  [Backup e Restauração](#)
-  [Performance](#)
-  [Suporte](#)

[object Object]



FlowChat API é a solução mais avançada para automação WhatsApp, desenvolvida com arquitetura moderna e recursos enterprise:

[object Object]

[object Object] •  Sessões Ilimitadas simultâneas •  QR Code dinâmico por sessão •  Reconnect automático •  Status em tempo real [object Object] •  OpenAI GPT nativo •  Streaming de respostas	[object Object] •  Grupos com controle total •  Webhooks com prioridades •  Message Collector automatizado •  Mídia com download automático [object Object] •  React 19 + Node.js •  MongoDB com fallback
---	---



-  **Tool Execution** avançada
 -  **Context** persistente
-  **Docker** otimizado
 -  **Proxy Reverso** ready

[object Object]

```
graph LR
  A[React Frontend] --> B[Express API]
  B --> C[Baileys Engine]
  B --> D[MongoDB]
  B --> E[OpenAI]
  C --> F[WhatsApp Web]
  B --> G[Webhook System]
```

Code

[object Object]

Tipo	Descrição	Benefícios
 Enterprise	Automação corporativa	Múltiplas sessões, IA integrada
 E-commerce	Atendimento automatizado	Webhooks, coleta de mensagens
 Educação	Suporte estudantil	AI assistant, grupos organizados
 Consultoria	Relacionamento cliente	Message collector, resumos IA

[object Object][object Object]

[object Object]

```
git clone https://github.com/seu-repo/flowchat-api
cd flowchat-api
```

Code

[object Object]

```
# Copie o exemplo
cp .env.example .env

# Configure as variáveis OBRIGATÓRIAS:
JWT_SECRET="sua-chave-jwt-super-secreta"
CORS_ORIGIN="https://seu-dominio.com"
OPENAI_API_KEY="sk-sua-chave-openai"
```

[object Object]

```
docker-compose up -d
# Pronto! 🚀 Acesse: http://localhost:8080
```

[object Object]

Passo	Ação	Resultado
1	Criar projeto no EasyPanel	✅ Projeto criado
2	Configurar variáveis de ambiente	✅ Segurança configurada
3	Deploy automático	✅ API em produção

💡 *Dica: Para deploy mais rápido, use nosso template EasyPanel pré-configurado!*

[object Object][object Object]

[object Object] • Ubuntu 20.04+ • Debian 11+ • CentOS 8+ • Rocky Linux 8+ • Amazon Linux 2	[object Object] • Windows 10/11 • Windows Server 2019+ • Docker Desktop • WSL2 (recomendado)	[object Object] • macOS 11+ • Docker Desktop • Intel/ARM (M1/M2) • Homebrew
---	---	--

[object Object]

Provider	Status	Template	Auto-Deploy
EasyPanel	✔ Recomendado	✔ Disponível	✔ 1-Click
Railway	✔ Suportado	✔ Disponível	✔ GitHub
DigitalOcean	✔ Suportado	🔑 Manual	📄 Droplet
AWS	✔ Suportado	🔑 Manual	📄 EC2/ECS
Google Cloud	✔ Suportado	🔑 Manual	📄 Compute

[object Object]

```
# Verificar versões mínimas
docker --version      # >= 20.10.0
docker-compose --version # >= 2.0.0

# Verificar recursos
docker system info | grep -E "CPUs|Total Memory"
```

Code

[object Object]

Configuração	CPU	RAM	Storage	Throughput
🔑 Desenvolvimento	1 vCore	1GB	5GB SSD	~10 msg/min
🚀 Produção Pequena	2 vCore	2GB	10GB SSD	~100 msg/min
🏢 Enterprise	4+ vCore	4GB+	50GB+ SSD	~1000+ msg/min

 **Importante:** MongoDB 4.4 funciona sem AVX (processadores mais antigos)

[object Object][object Object]

Code

```
flowchat-api/
├── docker-compose.yaml # Configuração principal do Docker
├── .env # Variáveis de ambiente (CRIAR)
├── .env.example # Exemplo de configuração
├── DEPLOYMENT_GUIDE.md # Esta documentação
├── pdf-converter.js # Conversor para PDF
├── volumes/ # Volumes Docker (auto-criados)
│   ├── sessions/ # Sessões WhatsApp persistentes
│   ├── .media/ # Arquivos de mídia baixados
│   ├── uploads/ # Uploads do usuário
│   ├── downloads/ # Downloads processados
│   ├── auth_sessions/ # Tokens de autenticação
│   └── logs/ # Logs da aplicação
└── containers/ # Containers Docker
    ├── baileys-api # API principal (porta 8080)
    └── mongodb # Banco de dados MongoDB
```

[object Object]

Diretório	Propósito	Auto-criado	Persistente
sessions/	Dados das sessões WhatsApp	✓	✓
.media/	Cache de mídia baixada	✓	✓
uploads/	Arquivos enviados via API	✓	✓
downloads/	Downloads processados	✓	✓
auth_sessions/	Tokens de autenticação	✓	✓
logs/	Logs da aplicação	✓	✓

[object Object][object Object]



```

services:
  # MongoDB Database
  mongodb:
    image: mongo:4.4
    restart: unless-stopped
    environment:
      MONGO_INITDB_ROOT_USERNAME: mongouser
      MONGO_INITDB_ROOT_PASSWORD: mongopassword
      MONGO_INITDB_DATABASE: baileys
    volumes:
      - baileys_mongodb:/data/db
    logging:
      driver: 'json-file'
      options:
        max-size: '100m'
        max-file: '10'
    healthcheck:
      test: ["CMD", "mongo", "--eval", "db.adminCommand('ping')"]
      interval: 30s
      timeout: 10s
      retries: 5
      start_period: 60s

  # Baileys WhatsApp API
  baileys-api:
    image: vinicius666/baileys-api:latest
    restart: unless-stopped
    environment:
      # [CONFIGURAR] Application Configuration
      NODE_ENV: production
      PORT: 3000
      TZ: America/Sao_Paulo

      # [CONFIGURAR] Database Configuration
      MONGODB_URI: mongodb://mongouser:mongopassword@mongodb:27017/baileys?authSource=admin
      DB_NAME: baileys

      # [CRÍTICO] Security Configuration - ALTERAR EM PRODUÇÃO!
      SESSION_SECRET: baileys-default-session-secret-change-in-production
      COOKIE_SECRET: baileys-default-cookie-secret-change-in-production
      JWT_SECRET: baileys-jwt-secret-change-in-production-with-strong-key
      JWT_EXPIRES_IN: 24h
      CSRF_SECRET: baileys-csrf-secret-change-in-production

      # [CONFIGURAR] CORS Configuration - ALTERAR PARA SEU DOMÍNIO
      CORS_ORIGIN: https://tg-flowchat.2nsqve.easypanel.host
      FRONTEND_URL: https://tg-flowchat.2nsqve.easypanel.host

      # [OPCIONAL] AI Assistant Configuration
      OPENAI_API_KEY: your-openai-api-key-here

      # [CONFIGURAR] API Token Configuration
      BAILEYS_API_TOKEN: baileys_production_api_token_change_this

      # [CONFIGURAR] WhatsApp Behavior Configuration
      AUTO_MARK_READ: true

      # [CONFIGURAR] MCP Server Configuration
      BASE_URL: https://tg-flowchat.2nsqve.easypanel.host
      BAILEYS_API_KEY: baileys_production_mcp_key_change_this
      API_TIMEOUT: 30000
      RATE_LIMIT: 100
    ports:

```

```

- "8080:3000"
volumes:
- './sessions:/app/.sessions'
- './.media:/app/.media'
- baileys_uploads:/app/uploads
- baileys_downloads:/app/downloads
- baileys_auth:/app/auth_sessions
- baileys_logs:/app/logs
depends_on:
  mongodb:
    condition: service_healthy
logging:
  driver: 'json-file'
  options:
    max-size: '100m'
    max-file: '10'
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:3000/api/management/health"]
  interval: 30s
  timeout: 10s
  retries: 3
  start_period: 60s

volumes:
  baileys_mongodb: {}
  baileys_uploads: {}
  baileys_downloads: {}
  baileys_auth: {}
  baileys_logs: {}

```

[object Object][object Object]

```

# Segurança JWT (CRÍTICO)
JWT_SECRET="gere-uma-chave-super-secreta-32-caracteres"
JWT_EXPIRES_IN="24h"

# Segurança Session & Cookies (CRÍTICO)
SESSION_SECRET="gere-uma-chave-session-super-secreta"
COOKIE_SECRET="gere-uma-chave-cookie-super-secreta"
CSRF_SECRET="gere-uma-chave-csrf-super-secreta"

# API Tokens (IMPORTANTE)
BAILEYS_API_TOKEN="baileys_seu_token_personalizado_aqui"
BAILEYS_API_KEY="baileys_sua_chave_mcp_personalizada"

```

 Code

[object Object]

```
# Substitua pelo SEU domínio
CORS_ORIGIN="https://seu-dominio.com"
FRONTEND_URL="https://seu-dominio.com"
BASE_URL="https://seu-dominio.com"

# Exemplo EasyPanel
CORS_ORIGIN="https://meuapp.xyz123.easypanel.host"
FRONTEND_URL="https://meuapp.xyz123.easypanel.host"
BASE_URL="https://meuapp.xyz123.easypanel.host"
```

[Code](#)

[object Object]

```
# OpenAI Assistant (opcional)
OPENAI_API_KEY="sk-sua-chave-openai-aqui"

# Comportamento WhatsApp
AUTO_MARK_READ="true"      # ou "false"
TZ="America/Sao_Paulo"    # seu timezone

# Performance
API_TIMEOUT="30000"        # 30 segundos
RATE_LIMIT="100"          # requisições por minuto
```

[Code](#)

[object Object]

```
# MongoDB - Manter como está
MONGODB_URI="mongodb://mongouser:mongopassword@mongodb:27017/baileys?authSource=admin"
DB_NAME="baileys"
NODE_ENV="production"
PORT="3000"
```

[Code](#)

[object Object][object Object]

1. Acesse seu painel EasyPanel
2. Clique em **"New Project"**
3. Escolha **"Git Repository"**
4. Cole a URL do repositório

[object Object]


```
# Build Settings
Build Command: echo "Using Docker Compose"
Start Command: docker-compose up -d
Port: 8080
```

[Code](#)

[object Object]

No painel EasyPanel, vá em **Environment Variables**:

```
# OBRIGATÓRIAS - Gere chaves únicas
JWT_SECRET=sua_chave_jwt_super_secreta_32_chars
SESSION_SECRET=sua_chave_session_super_secreta
COOKIE_SECRET=sua_chave_cookie_super_secreta
CSRF_SECRET=sua_chave_csrf_super_secreta

# DOMÍNIO - Substitua pelo domínio fornecido pelo EasyPanel
CORS_ORIGIN=https://seu-app.xyz123.easypanel.host
FRONTEND_URL=https://seu-app.xyz123.easypanel.host
BASE_URL=https://seu-app.xyz123.easypanel.host

# API TOKENS - Customize
BAILEYS_API_TOKEN=baileys_sua_chave_personalizada
BAILEYS_API_KEY=baileys_sua_chave_mcp_personalizada

# OPCIONAL - Se quiser AI Assistant
OPENAI_API_KEY=sk-sua-chave-openai
```

[Code](#)

[object Object]

1. Clique em **"Deploy"**
2. Aguarde o build completar
3. Verifique se os containers estão rodando
4. Teste o acesso pelo domínio fornecido

[object Object][object Object]

Se você tem um domínio próprio:

```
# 1. Configure DNS
A record: seu-dominio.com → IP-DO-SERVIDOR
CNAME: www.seu-dominio.com → seu-dominio.com

# 2. Atualize variáveis
CORS_ORIGIN=https://seu-dominio.com
FRONTEND_URL=https://seu-dominio.com
BASE_URL=https://seu-dominio.com
```

[Code](#)

[object Object]

O EasyPanel configura SSL automaticamente. Para servidores próprios:

```
# Certbot (Let's Encrypt)
sudo certbot --nginx -d seu-dominio.com

# Ou configure um proxy reverso (nginx/caddy)
```

[Code](#)

[object Object][object Object]

```
# ✅ Alterar TODAS as chaves secretas
JWT_SECRET="✅ ALTERADO"
SESSION_SECRET="✅ ALTERADO"
COOKIE_SECRET="✅ ALTERADO"
CSRF_SECRET="✅ ALTERADO"

# ✅ Configurar CORS corretamente
CORS_ORIGIN="✅ SEU DOMÍNIO"

# ✅ API Tokens únicos
BAILEYS_API_TOKEN="✅ PERSONALIZADO"

# ✅ Configurar firewall (se servidor próprio)
# Permitir apenas portas 80, 443, 8080

# ✅ Backup automático
# Configure backup dos volumes Docker
```

[Code](#)

[object Object]

```
# Linux/Mac - gerar chaves de 32 caracteres
openssl rand -base64 32

# Windows PowerShell
[System.Web.Security.Membership]::GeneratePassword(32, 4)

# Online (seguro)
# https://www.allkeysgenerator.com/Random/Security-Encryption-Key-Generator.aspx
```

[Code](#)

[object Object][object Object]

```
# Verificar status dos containers
docker-compose ps

# Logs da aplicação
docker-compose logs baileys-api

# Logs do MongoDB
docker-compose logs mongodb

# Health check manual
curl http://localhost:8080/api/management/health
```

[Code](#)

[object Object]

- **CPU Usage:** < 80%
- **Memory Usage:** < 80%
- **Disk Space:** > 2GB livres
- **MongoDB Connection:** Healthy
- **WhatsApp Sessions:** Connected

[object Object][object Object][object Object]

```
# Erro: Bind for :::8080 failed: port is already allocated
# Solução: Alterar porta no docker-compose.yaml
ports:
  - "8081:3000" # Use porta diferente
```

[Code](#)

[object Object]

```
# Erro: dependency failed to start: container mongodb is unhealthy
# Solução: CPU sem AVX - MongoDB 4.4 já configurado
# Aguardar mais tempo ou verificar logs
docker-compose logs mongodb
```

[Code](#)

[object Object]

```
# Erro: blocked by CORS policy
# Solução: Verificar CORS_ORIGIN nas variáveis de ambiente
CORS_ORIGIN=https://SEU-DOMINIO-EXATO.com
```

[Code](#)

[object Object]

```
# Erro: MongoDB connection failed
# Solução: Aguardar MongoDB inicializar completamente
docker-compose restart baileys-api
```

[Code](#)

[object Object]

```
# Problema: QR Code não aparece
# Solução:
1. Verificar logs: docker-compose logs baileys-api
2. Acessar: https://seu-dominio.com/api/baileys/
3. Criar nova sessão via API
```

[Code](#)

[object Object]

```
# Ver logs em tempo real
docker-compose logs -f baileys-api

# Logs de erro do MongoDB
docker-compose logs mongodb | grep ERROR

# Status detalhado dos containers
docker-compose ps
docker inspect container_name
```

[Code](#)

[object Object][object Object][object Object]

```
GET /api/management/health # Health check
POST /api/management/auth/register # Registro de usuário
POST /api/management/auth/login # Login
GET /api/management/auth/profile # Perfil do usuário
```

[Code](#)

[object Object]

```
GET /api/baileys/sessions # Listar sessões
POST /api/baileys/sessions/{sessionId}/start # Iniciar sessão
DELETE /api/baileys/sessions/{sessionId} # Deletar sessão
GET /api/baileys/sessions/{sessionId}/qr # QR Code
POST /api/baileys/sessions/{sessionId}/send-message # Enviar mensagem
```

[Code](#)

[object Object]

```
GET    /api/baileys/groups/{sessionId}/list      # Listar grupos
POST   /api/baileys/groups/{sessionId}/create    # Criar grupo
POST   /api/baileys/groups/{sessionId}/{groupId}/add # Adicionar membros
DELETE /api/baileys/groups/{sessionId}/{groupId}/kick # Remover membros
```

[Code](#)

[object Object]

```
POST /api/management/ai/chat      # Chat com AI
GET   /api/management/ai-agents/list # Listar agentes
POST  /api/management/ai-agents/create # Criar agente
```

[Code](#)

[object Object]

```
GET    /api/message-collector/tasks      # Listar tarefas
POST   /api/message-collector/tasks      # Criar tarefa
GET    /api/message-collector/tasks/{taskId}/messages # Ver mensagens
DELETE /api/message-collector/tasks/{taskId} # Deletar tarefa
```

[Code](#)

[object Object]

Acesse a documentação Swagger completa em:

```
https://seu-dominio.com/api-docs
```

[Code](#)

[object Object][object Object]

- 📖 **Documentação:** [/api-docs](#)
- 🐞 **Issues:** [GitHub Issues](#)
- 💬 **Comunidade:** Discord/Telegram
- 📧 **Suporte:** suporte@seudominio.com

[object Object]

```
# Versão da aplicação
curl https://seu-dominio.com/api/management/health

# Versão do Docker
docker --version
docker-compose --version

# Versão do MongoDB
docker-compose exec mongodb mongo --version
```

[Code](#)

[object Object][object Object]

```
# Backup dos volumes Docker
docker run --rm -v baileys_mongodb:/data -v $(pwd):/backup alpine tar czf /backup/mongodb-backup.tar.gz -
docker run --rm -v baileys_uploads:/data -v $(pwd):/backup alpine tar czf /backup/uploads-backup.tar.gz -
```

[Code](#)

[object Object]

```
# Atualizar imagem
docker-compose pull
docker-compose up -d

# Ver logs após atualização
docker-compose logs -f baileys-api
```

[Code](#)

[object Object]

Para melhor performance:

1. **Usar SSD** para storage
2. **Mínimo 2GB RAM** para produção
3. **Conexão estável** para WhatsApp
4. **Monitorar logs** regularmente
5. **Backup automático** configurado

[object Object][object Object]

Configure o AI Assistant para recursos avançados:

```
# Obter API Key OpenAI
# 1. Acesse: https://platform.openai.com/api-keys
# 2. Clique "Create new secret key"
# 3. Copie a chave (sk-...)
OPENAI_API_KEY="sk-proj-sua-chave-aqui"
```

[object Object]

Modelo	Capability	Custo	Recomendado para
gpt-4	★★★★★	Alto	Produção Enterprise
gpt-4-turbo	★★★★★	Médio	Produção Padrão
gpt-3.5-turbo	★★★	Baixo	Desenvolvimento/Teste

[object Object][object Object]

```
#!/bin/bash
# backup-flowchat.sh

DATE=$(date +%Y%m%d_%H%M%S)
BACKUP_DIR="./backups/$DATE"

# Criar diretório de backup
mkdir -p $BACKUP_DIR

# Backup MongoDB
docker-compose exec -T mongodb mongodump --archive > $BACKUP_DIR/mongodb_$DATE.archive

# Backup volumes
docker run --rm -v baileys_sessions:/data -v $PWD/$BACKUP_DIR:/backup alpine tar czf /backup/sessions_$DATE.tar.gz /data
docker run --rm -v baileys_uploads:/data -v $PWD/$BACKUP_DIR:/backup alpine tar czf /backup/uploads_$DATE.tar.gz /data

echo "✅ Backup concluído: $BACKUP_DIR"
```

[object Object]

```
#!/bin/bash
# restore-flowchat.sh

BACKUP_DATE=$1
BACKUP_DIR="./backups/$BACKUP_DATE"

# Restaurar MongoDB
docker-compose exec -T mongodb mongorestore --archive < $BACKUP_DIR/mongodb_$BACKUP_DATE.archive

# Restaurar volumes
docker run --rm -v baileys_sessions:/data -v $PWD/$BACKUP_DIR:/backup alpine tar xzf /backup/sessions_$BACKUP_DATE.tar.gz -C /data
docker run --rm -v baileys_uploads:/data -v $PWD/$BACKUP_DIR:/backup alpine tar xzf /backup/uploads_$BACKUP_DATE.tar.gz -C /data

echo "✅ Restauração concluída!"
```

[object Object][object Object][object Object]

```
# /etc/nginx/sites-available/flowchat
server {
    listen 80;
    server_name seu-dominio.com;

    # Rate limiting
    limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s;

    location / {
        limit_req zone=api burst=20 nodelay;

        proxy_pass http://localhost:8080;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;

        # Timeouts
        proxy_connect_timeout 60s;
        proxy_send_timeout 60s;
        proxy_read_timeout 60s;
    }

    # Static files caching
    location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }
}
```

[object Object]

Code

```
# docker-compose.production.yml
services:
  baileys-api:
    deploy:
      resources:
        limits:
          cpus: '2.0'
          memory: 2G
        reservations:
          cpus: '1.0'
          memory: 1G

  mongodb:
    deploy:
      resources:
        limits:
          cpus: '1.0'
          memory: 1G
        reservations:
          cpus: '0.5'
          memory: 512M
```

[object Object]

Code

```
# Production optimizations
NODE_ENV="production"
NODE_OPTIONS="--max-old-space-size=2048"

# MongoDB optimization
MONGODB_URI="mongodb://user:pass@host:27017/db?maxPoolSize=50&w=majority&retryWrites=true"

# Performance settings
API_TIMEOUT="60000"      # 60 segundos
RATE_LIMIT="200"         # 200 req/min
MAX_CONNECTIONS="100"    # 100 conexões simultâneas
```

[object Object]

Métrica	Desenvolvimento	Produção	Enterprise
Latência média	150ms	50ms	25ms
Throughput	10 msg/min	500 msg/min	5000+ msg/min
Sessões simultâneas	5	50	500+
Uptime	95%	99.5%	99.9%

[object Object][object Object]

[object Object] 📖 Documentação Oficial 🔍 Swagger/OpenAPI 📄 Guias e Tutoriais ❓ FAQ [object Object] 💬 Discord Server 📠 Telegram Group 🐦 Twitter 📺 YouTube	[object Object] ✉️ Email: suporte@flowchat.api 🎫 Tickets: Support Portal ☎️ Urgente: +55 (11) 99999-9999 🔴 Status: status.flowchat.api [object Object] 👤 Account Manager: enterprise@flowchat.api 📝 Consultoria: consulting@flowchat.api 🛡️ Security: security@flowchat.api ⚖️ Legal/Compliance: legal@flowchat.api
--	--

[object Object]

Plano	Response Time	Uptime	Support
 Community	48h	95%	Discord/Telegram
 Professional	12h	99.5%	Email + Priority
 Enterprise	2h	99.9%	24/7 + Dedicated

[object Object]

Code

```
# Verificar versões
curl https://seu-dominio.com/api/management/health

# Exemplo de resposta
{
  "status": "healthy",
  "version": "2.4.1",
  "build": "2025.08.26.001",
  "uptime": "7d 12h 34m",
  "environment": "production",
  "features": {
    "ai_assistant": true,
    "multi_session": true,
    "webhooks": true,
    "message_collector": true
  }
}
```

[object Object]

 Deploy Success [object Object]

[object Object]

- ☐ **Variáveis configuradas** - Todas as **[CONFIGURAR]** e **[CRÍTICO]**
- ☐ **Domínio configurado** - CORS_ORIGIN atualizado
- ☐ **Segurança implementada** - Chaves únicas geradas
- ☐ **Backup configurado** - Scripts de backup funcionando
- ☐ **Monitoramento ativo** - Health checks operacionais
- ☐ **Performance otimizada** - Recursos dimensionados
- ☐ **Documentação lida** - Swagger e guias consultados

[object Object]

1. **Configure seu primeiro bot** via **/api/baileys/sessions**
2. **Teste o AI Assistant** no painel web
3. **Configure webhooks** para integração
4. **Monte estratégia de backup** automatizado
5. **Monitore performance** e ajuste recursos

[object Object]

-  **Painel Admin:** **<https://seu-dominio.com>**
-  **API Docs:** **<https://seu-dominio.com/api-docs>**
-  **Health Check:** **<https://seu-dominio.com/api/management/health>**
-  **Primeira Sessão:** **<https://seu-dominio.com/api/baileys/sessions>**

Desenvolvido com  pela equipe FlowChat

[GitHub](#) [Repository](#)

[Docker](#) [Hub](#)

[License](#) [MIT](#)

© 2025 FlowChat API. Todos os direitos reservados.

 **Importante:** Altere **TODAS** as variáveis marcadas como **[CONFIGURAR]** e **[CRÍTICO]** antes de usar em produção!